Project 2 – Software Security Analysis

Code Analysis in Ghidra, Finding Bugs and Applying a Patch to Code Binaries, and Vulnerability Assessment using OWASP ZAP Proxy

Introduction

This project is designed to focus on the analysis of software security through reverse engineering and vulnerability assessment techniques. The main goal is to understand how compiled binaries operate without access to source code and identify potential flaws that may affect system functionality and security. During this project, tools like Ghidra and IDA Pro were used to analyze executable files. These tools are used for the examination of program structure, functions, and logic at both assembly and pseudocode levels. Using this process, we can reconstruct the internal behavior of the programs to identify logical errors and implementation issues. The second part of the project includes finding the problem and applying patches to a binary to fix identified bugs. We can modify assembly instructions to correct the program error.

In the third part of the project started by downloading and installing OWASP ZAP (Zed Attack Proxy) in a Kali VM. This tool is used to perform automated scanning of a test website to identify security weaknesses like missing security headers, improper input validation, and other common web vulnerabilities.

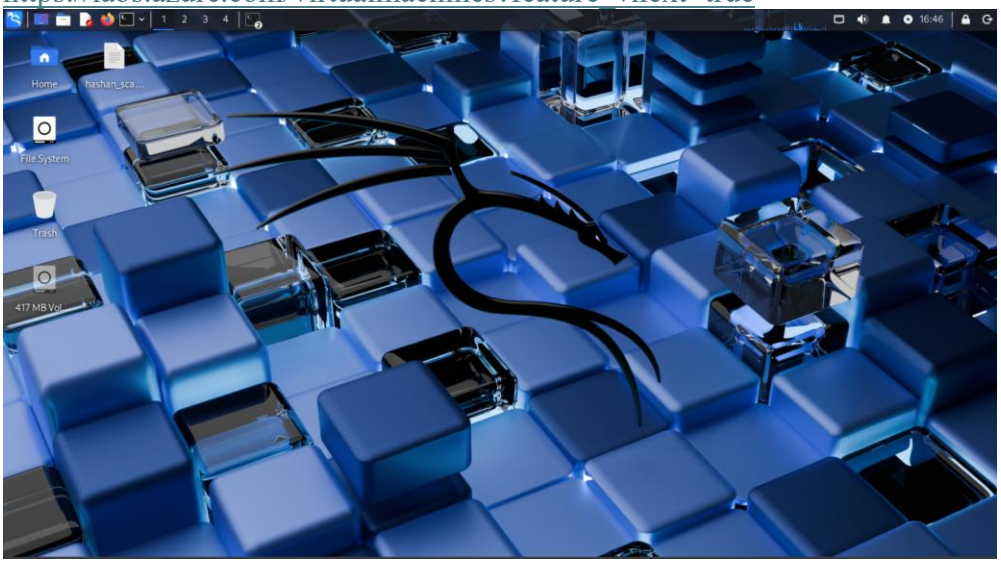
This project provides a comprehensive understanding of both low-level program analysis and high-level web security assessment. This work highlights the importance of secure coding practices and proactive vulnerability mitigation.

Part 1 - Code Analysis in Ghidra

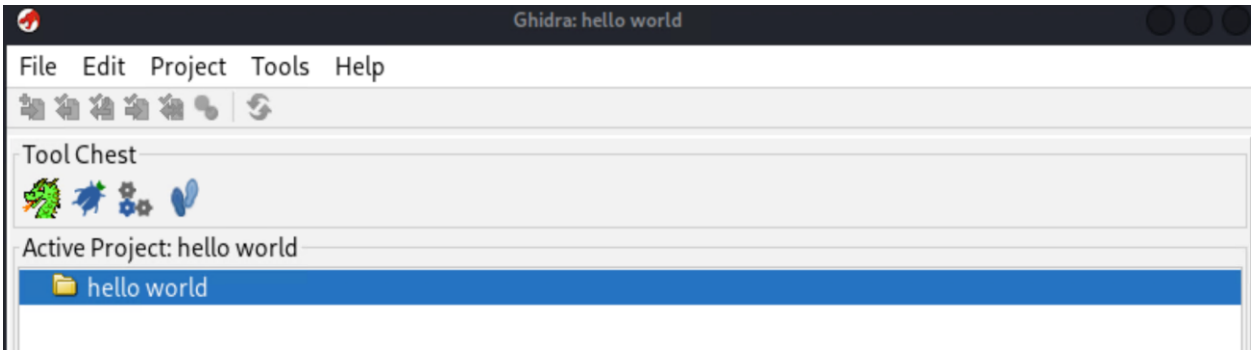
Objective - To demonstrate the ability to analyze binaries using the tool Ghidra.

Part 1a

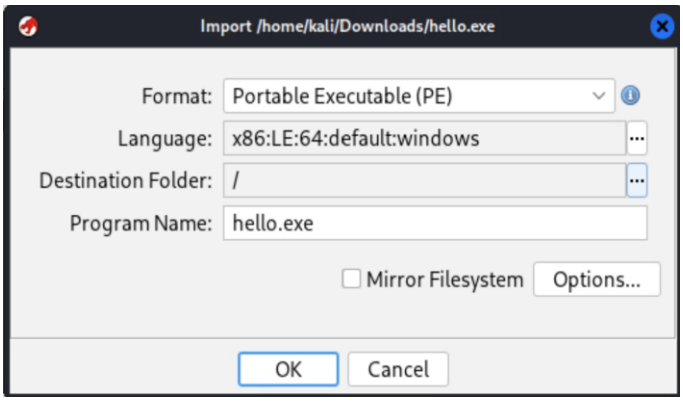
1. Logged to Azure Lab environment using following web link and logged into the Kali VM using Oracle VirtualBox.
https://labs.azure.com/virtualmachines?feature_vnext=true



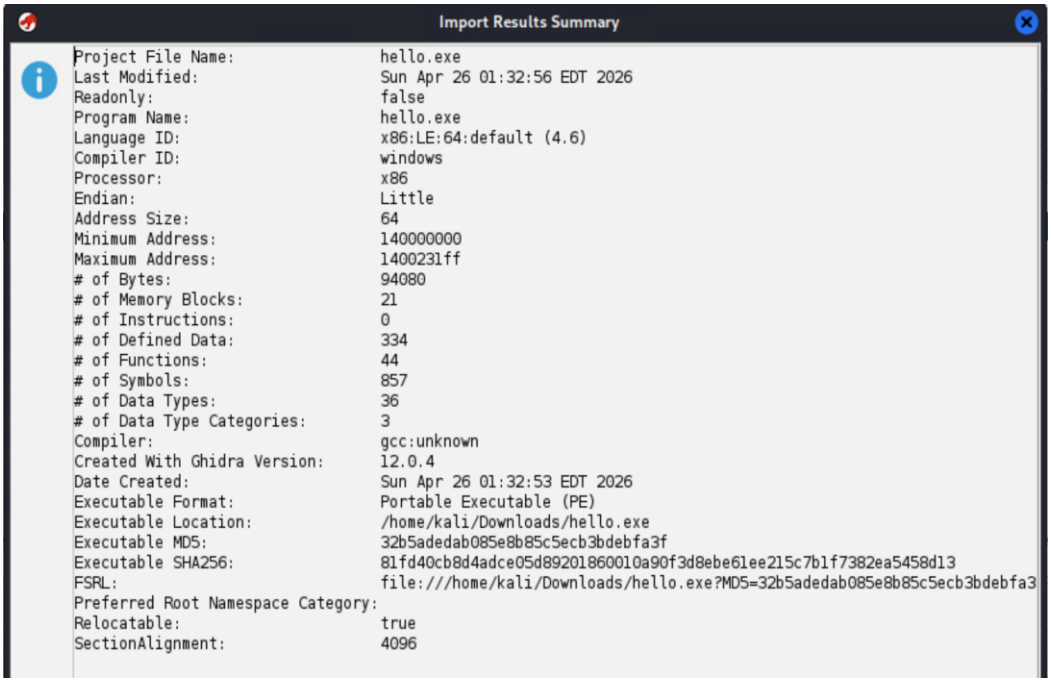
2. Ghidra was launched within a Kali Linux virtual machine environment and a new non-shared project named *hello world* was created to organize the analysis.



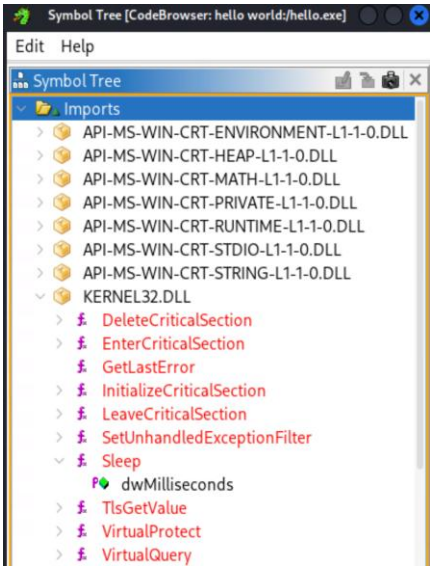
3. The binary file *hello.exe* was imported into Ghidra using the import dialog.



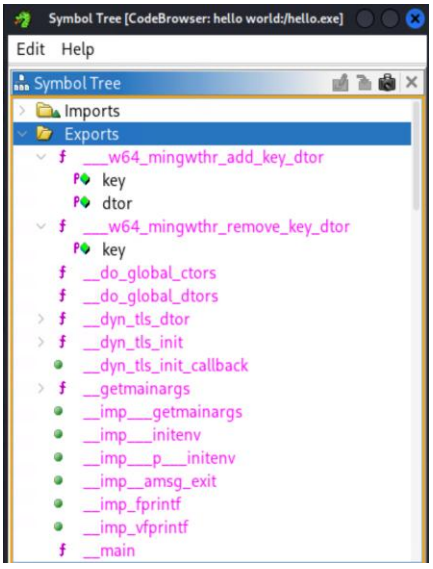
4. After importing the binary, Ghidra generated an import results summary containing detailed information about the executable.



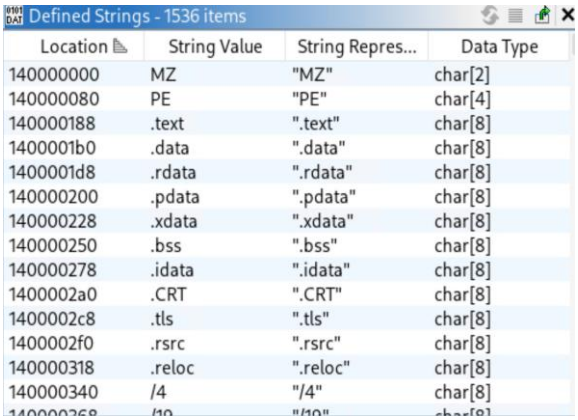
5. I selected *KERNEL32.dll* from *Imports*. The *GetLastError* function is used to retrieve error codes from the system, which can help identify issues during program execution. The function *Sleep* is used to pause the execution of the program for a specified number of milliseconds.



6. I selected *_do_global_ctors* and *_do_global_dtors* functions from *Exports*. The *_do_global_ctors* function is responsible for initializing global constructors before the main function executes. The *_do_global_dtors* function is used to handle the cleanup and destruction of global objects after program execution.



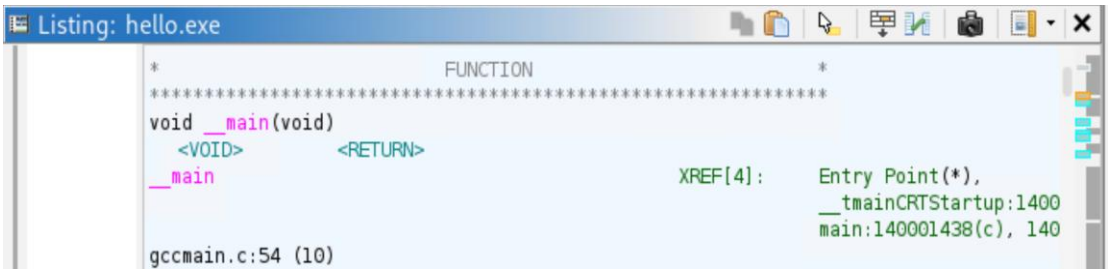
7. The strings reveal information about their structure. The *.data* represents the section of the executable that stores initialized global and static variables used during program execution. The string *.rsrc* refers to the resource section of the executable, which typically contains embedded resources such as icons, menus, or other application data.



Defined Strings - 1536 items

Location	String Value	String Repres...	Data Type
140000000	MZ	"MZ"	char[2]
140000080	PE	"PE"	char[4]
140000188	.text	".text"	char[8]
1400001b0	.data	".data"	char[8]
1400001d8	.rdata	".rdata"	char[8]
140000200	.pdata	".pdata"	char[8]
140000228	.xdata	".xdata"	char[8]
140000250	.bss	".bss"	char[8]
140000278	.idata	".idata"	char[8]
1400002a0	.CRT	".CRT"	char[8]
1400002c8	.tls	".tls"	char[8]
1400002f0	.rsrc	".rsrc"	char[8]
140000318	.reloc	".reloc"	char[8]
140000340	/4	"/4"	char[8]

7. The entry point of the program is the main function (_main) which is located at 0x140001438. This function represents the starting point of execution.

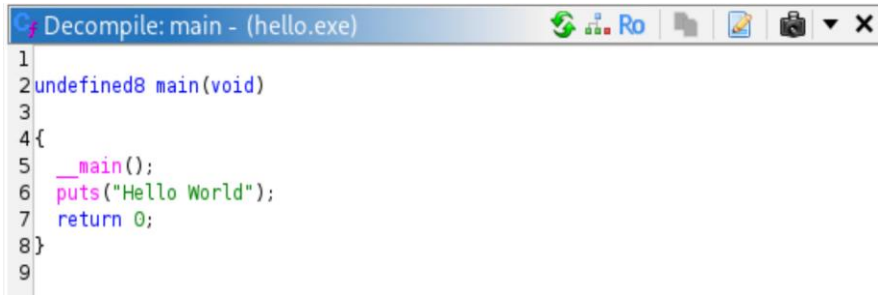


Listing: hello.exe

```
*                          FUNCTION                          *
*************************************************************
void __main(void)
  <VOID>        <RETURN>
__main                                  XREF[4]:    Entry Point(*),
                                                    __tmainCRTStartup:1400
                                                    main:140001438(c), 140

gccmain.c:54 (10)
```

8. The program exits within the main function through a *return 0* statement. The decompiled code shows that the function returns after executing initialization logic, indicating the termination point of the program.

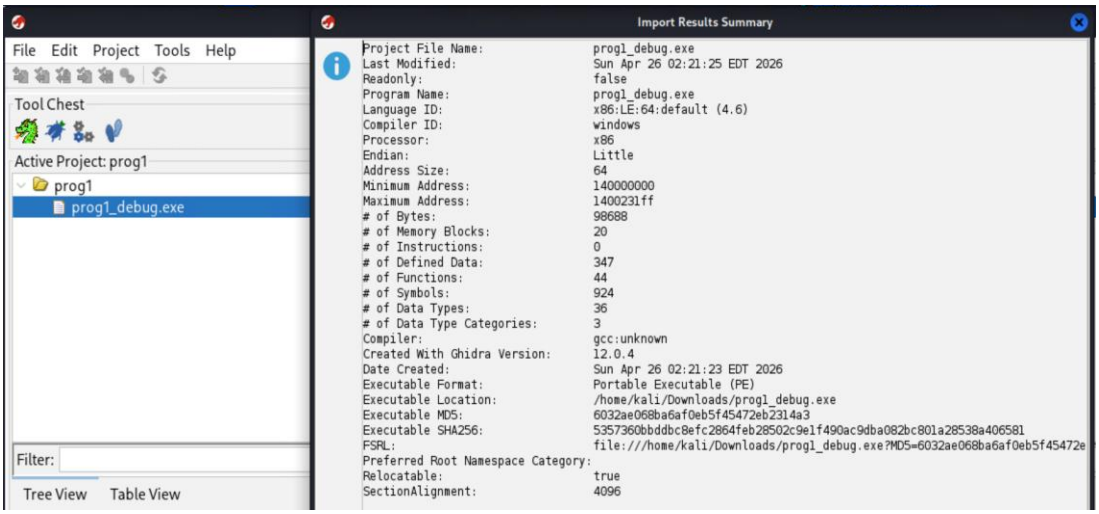


Decompile: main - (hello.exe)

```
1
2 undefined8 main(void)
3
4 {
5   __main();
6   puts("Hello World");
7   return 0;
8 }
9
```

Part 1b

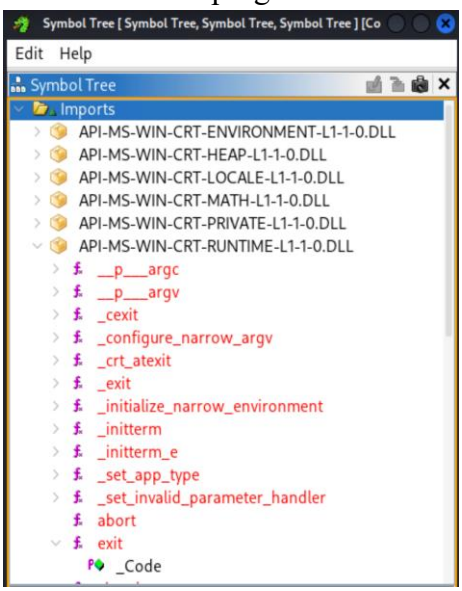
1. The binary file proj1_debug.exe was imported into Ghidra. The executable was successfully loaded, and automatic analysis was performed using default settings.



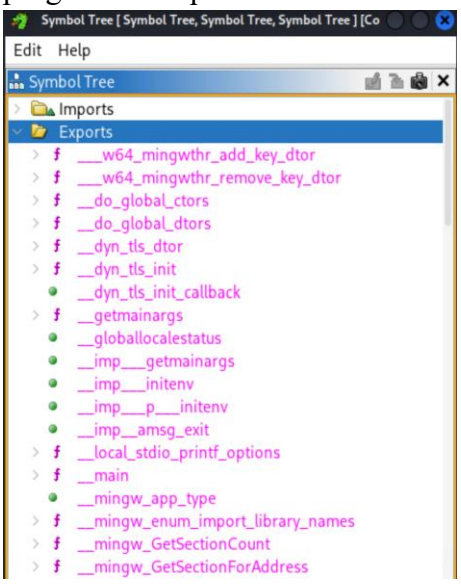
The file properties of *proj1_debug.exe* were identified using Ghidra's import results summary.

Program Name - proj1_debug.exe
Size - 98,688 bytes
MD5 Hash - 6032ae068ba6af0eb5f45472eb2314a3
SHA256 Hash -5357360bbddbc8efc2864feb28502c9e1f490ac9dba082bc801a28538a406581
File Type - Portable Executable (PE)

2. Captured the binary imports functions from the C runtime library. The *abort* function is used to immediately terminate the program when a critical error occurs. The *exit* function is used to terminate the program and return control to the operating system in a controlled manner.



3. I selected *_getmainargs* and *_dyn_tls_init_callback* functions from *Exports*. The function *_getmainargs* is responsible for retrieving and processing command-line arguments passed to the program. The function *_dyn_tls_init_callback* is used to initialize thread-local storage during program startup.



4. The strings reveal information about their structure. The *MZ* string represents the standard header signature of a Windows Portable Executable (PE) file, confirming the file format. The string *.text* refers to the section of the executable that contains the program's executable code.

Defined Strings - 1623 items

Locati...	String Value	String Repre...	Data Type
140000000	MZ	"MZ"	char[2]
140000080	PE	"PE"	char[4]
140000188	.text	".text"	char[8]
1400001b0	.data	".data"	char[8]
1400001d8	.rdata	".rdata"	char[8]
140000200	.pdata	".pdata"	char[8]
140000228	.xdata	".xdata"	char[8]
140000250	.bss	".bss"	char[8]
140000278	.idata	".idata"	char[8]
1400002a0	.tls	".tls"	char[8]
1400002c8	.rsrc	".rsrc"	char[8]
1400002f0	.reloc	".reloc"	char[8]
140000318	/4	"/4"	char[8]
140000340	/19	"/19"	char[8]

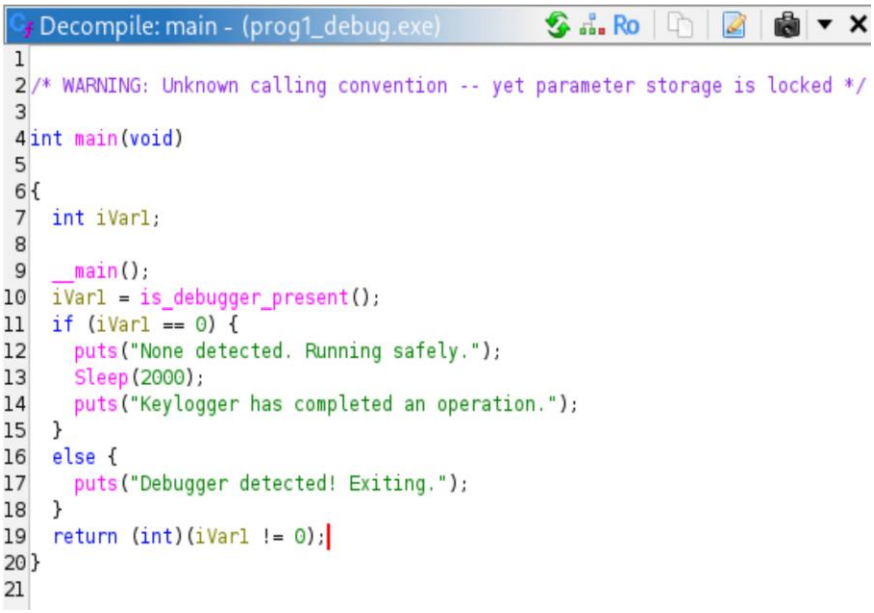
5. What anti-reverse engineering techniques does the binary employ?

The binary employs an anti-debugging technique by calling the function *is_debugger_present()*. This function is going to check whether the program is being executed under a debugger. If a debugger is detected, then the program terminates immediately. This kind of behavior is designed to prevent reverse engineering and analysis by security tools such as

Ghidra or *IDA Pro*. Also, the conditional logic based on debugger detection makes it harder to analyze the program's normal execution flow.

6. What is the code designed to do?

This code is designed to detect whether a debugger is attached and responds accordingly. If no debugger is detected, then the program prints the message *None detected. Running safely.* After that it waits for a short period using the Sleep function and then prints *Keylogger has completed an operation*. Also, if a debugger is detected, the program prints *Debugger detected! Exiting.* and terminates the execution. When analyzing the program, this program is attempting to hide its true functionality. It's possibly a keylogger which is a common technique used in malicious or protected software.

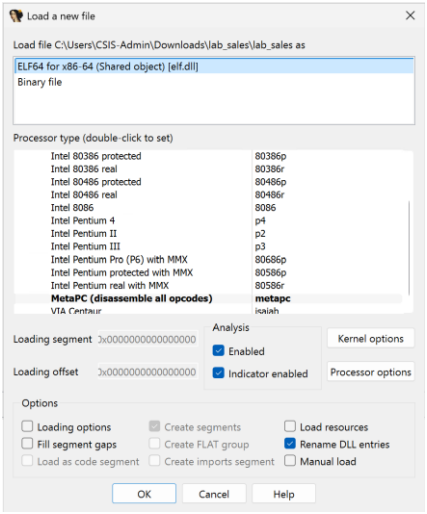


Part 2 - Finding Bugs and Applying a Patch to code Binaries

Objective: To perform software security analysis and apply a fix to buggy code binaries.

Part 2a - Finding the Problem

1. Launched IDA Pro (free) and opened the compiled binary as “ELF64…”.



2. Captured a screenshot of the Functions Window. The program contains approximately 24 functions, including both internal functions and external library functions

Function name	Segment	Start	Length	Locals	Arguments	R	F	L	M	O	S	B	T	=	X
_init_proc	.init	0000000000001000	0000001B	00000008		R	.	.	.	.	.	.	.	.	.
sub_1020	.plt	0000000000001020	0000000C			R	.	.	.	.	.	.	.	.	.
sub_1030	.plt	0000000000001030	0000000E			R	.	.	.	.	.	.	.	.	.
sub_1040	.plt	0000000000001040	0000000E			R	.	.	.	.	.	.	.	.	.
sub_1050	.plt	0000000000001050	0000000E			R	.	.	.	.	.	.	.	.	.
__cxa_finalize	**.plt.got**	**0000000000001060**	**0000000A**			**R**	.	.	.	.	.	.	**T**	.	.
__stack_chk_fail	.plt.sec	0000000000001070	0000000A			.	.	.	.	.	.	.	.	.	.
_printf	**.plt.sec**	**0000000000001080**	**0000000A**			**R**	.	.	.	.	.	.	**T**	.	.
__isoc99_scanf	.plt.sec	0000000000001090	0000000A			R	.	.	.	.	.	.	.	.	.
_start	.text	00000000000010A0	00000026			.	.	.	.	.	.	.	.	.	.
deregister_tm_clones	.text	00000000000010D0	00000029	00000000		R	.	.	.	.	.	.	.	.	.
register_tm_clones	.text	0000000000001100	00000039	00000000		R	.	.	.	.	.	.	.	.	.
__do_global_dtors_aux	.text	0000000000001140	00000039	00000000		R	.	.	.	.	.	.	.	.	.
frame_dummy	.text	0000000000001180	00000009	00000000		R	.	.	.	.	.	.	.	.	.
main	**.text**	**0000000000001189**	**000000E4**	**00000028**		**R**	.	.	.	.	.	**B**	**T**	.	.
_term_proc	.fini	0000000000001270	0000000D	00000008		R	.	.	.	.	.	.	.	.	.
__libc_start_main	**extern**	**0000000000004020**	**00000008**			**R**	.	.	.	.	.	.	**T**	.	.
__stack_chk_fail	extern	0000000000004028	00000008			.	.	.	.	.	.	.	.	.	.
printf	**extern**	**0000000000004030**	**00000008**			**R**	.	.	.	.	.	.	**T**	.	.
__isoc99_scanf	extern	0000000000004038	00000008			R	.	.	.	.	.	.	.	.	.
__imp___cxa_finalize	**extern**	**0000000000004040**	**00000008**			**R**	.	.	.	.	.	.	**T**	.	.
_ITM_deregisterTMCloneTable	extern	0000000000004048	00000008			R	.	.	.	.	.	.	.	.	.
__gmon_start__	extern	0000000000004050	00000008			R	.	.	.	.	.	.	.	.	.
_ITM_registerTMCloneTable	extern	0000000000004058	00000008			R	.	.	.	.	.	.	.	.	.

3. I analyzed key functions in pseudocode which are relevant to program execution. These include the *main* function, which contains the core logic, and supporting functions such as start, and _init_proc which handle program initialization and termination.

main function in pseudocode – The main function is handling the core functionality of the program. It prompts the user to enter the amount purchased, reads the input using *scanf*, and calculates the sales tax at a rate of 7.5%. In here, the total bill is wrongly calculated by subtracting the tax from

the original amount instead of adding it. The program then prints both the sales tax and the incorrect total bill.

v6 = v5[0] - v3 is wrong and should be v6 = v5[0] + v3;

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  float v3; // xmm0_4
  float v5[2]; // [rsp+Ch] [rbp-14h] BYREF
  float v6; // [rsp+14h] [rbp-Ch]
  unsigned __int64 v7; // [rsp+18h] [rbp-8h]

  v7 = __readfsqword(0x28u);
  printf("\nEnter in the amount purchased: ");
  __isoc99_scanf("%f", v5);
  v3 = 0.075 * v5[0];
  v5[1] = v3;
  v6 = v5[0] - v3;
  printf("\nThe sales tax is $%4.2f", v3);
  printf("\nThe total bill is $%5.2f\n", v6);
  return 0;
}
```

_start function in pseudocode – start function is the entry point of the program. It initializes the execution environment and calls the main function using the *__libc_start_main* routine.

```
1  // positive sp value has been detected, the output may be wrong!
2  void __fastcall __noreturn start(__int64 a1, __int64 a2, void (*a3)())
3  {
4    __int64 v3; // rax
5    int v4; // esi
6    __int64 v5; // [rsp-8h] [rbp-8h] BYREF
7    char *retaddr; // [rsp+0h] [rbp+0h] BYREF
8
9    v4 = v5;
10   v5 = v3;
11   _libc_start_main((int (*)(int, char **, char **))main, v4, &retaddr, nullptr, nullptr, a3, &v5);
12   __halt();
13 }
```

_int_proc function in pseudocode - The _init_proc function is responsible for initializing the program before execution.

```
1  // Alternative name is '_init'
2  __int64 (**init_proc())(void)
3  {
4    __int64 (**result)(void); // rax
5
6    result = &_gmon_start__;
7    if ( &_gmon_start__ )
8      return (__int64 (**)(void))_gmon_start__();
9    return result;
10 }
```

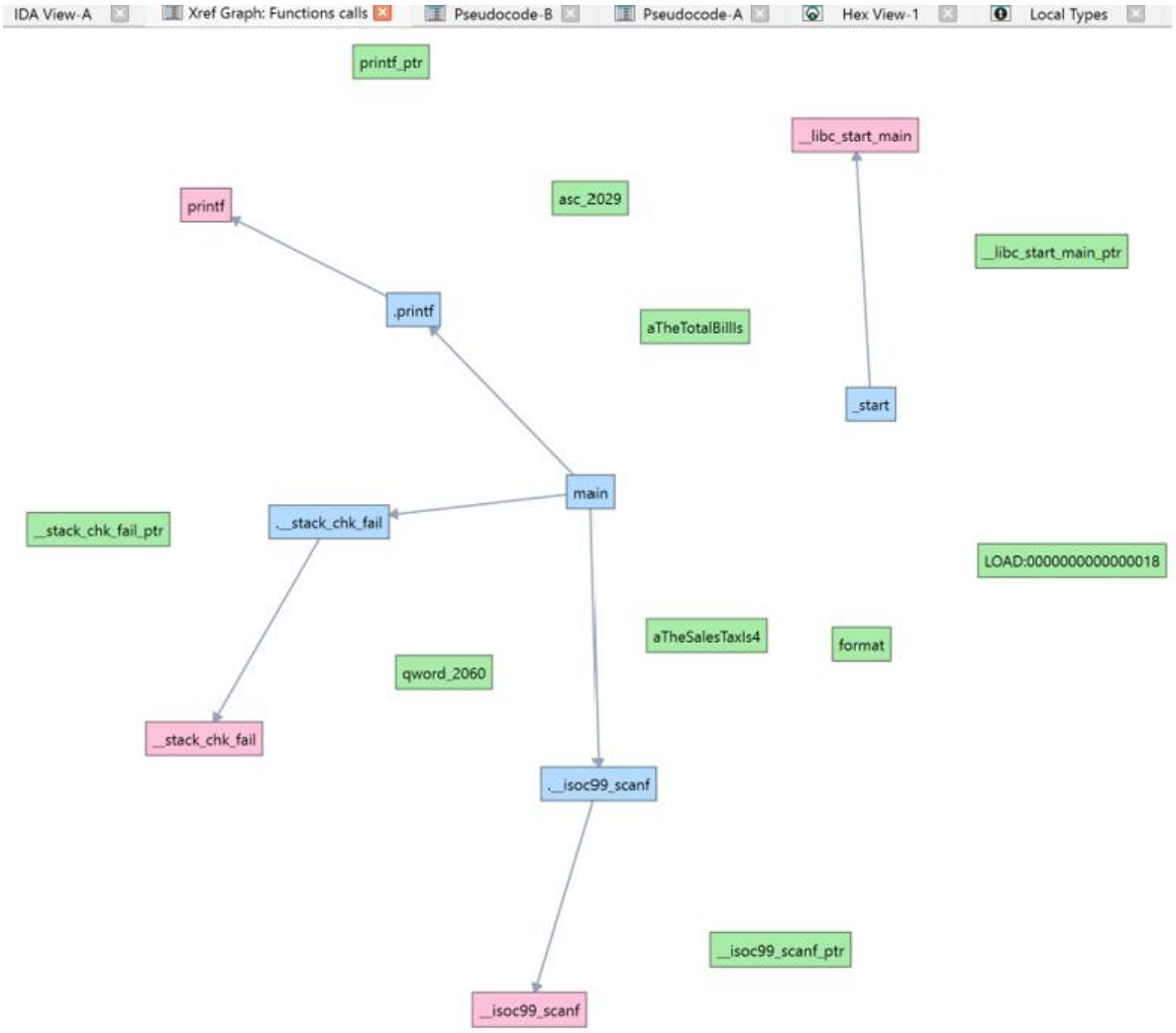
4. Opened the strings functions in the program by navigating to *View >Open Subviews >Strings.* This reveals several important strings that describe the functionality of the program including *Enter in the amount purchased, The sales tax is $%4.2f, The total bill is $%5.2f.*

The strings show that the program asks for input, calculates tax, and prints results. This confirms the purpose of the program. However, the strings do not provide any information about how the total bill is computed. This suggests a possible error in the calculation logic.

Address	Length	Type	String
LOAD:00000...	0000001C	C	/lib64/ld-linux-x86-64.so.2
LOAD:00000...	00000011	C	__stack_chk_fail
LOAD:00000...	00000012	C	__libc_start_main
LOAD:00000...	0000000F	C	__cxa_finalize
LOAD:00000...	00000007	C	printf
LOAD:00000...	0000000F	C	__isoc99_scanf
LOAD:00000...	0000000A	C	libc.so.6
LOAD:00000...	0000000A	C	GLIBC_2.7
LOAD:00000...	0000000C	C	GLIBC_2.2.5
LOAD:00000...	0000000A	C	GLIBC_2.4
LOAD:00000...	0000000B	C	GLIBC_2.34
LOAD:00000...	0000001C	C	_ITM_deregisterTMCloneTable
LOAD:00000...	0000000F	C	__gmon_start__
LOAD:00000...	0000001A	C	_ITM_registerTMCloneTable
.rodata:0000...	00000021	C	\nEnter in the amount purchased:
.rodata:0000...	00000019	C	\nThe sales tax is $%4.2f
.rodata:0000...	0000001B	C	\nThe total bill is $%5.2f\n
.eh_frame:0...	00000006	C	9*3$\"

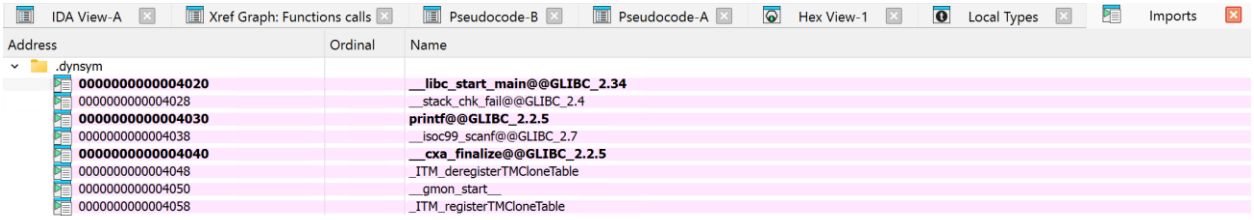
5. The function call graph shows the main function is responsible for both input handling and output generation. But the graph does not show any additional function dedicated to calculating the total bill.

It suggests that the computation is performed directly within the main function. validation of the computation process. If there is a bug, it is likely in the main function.

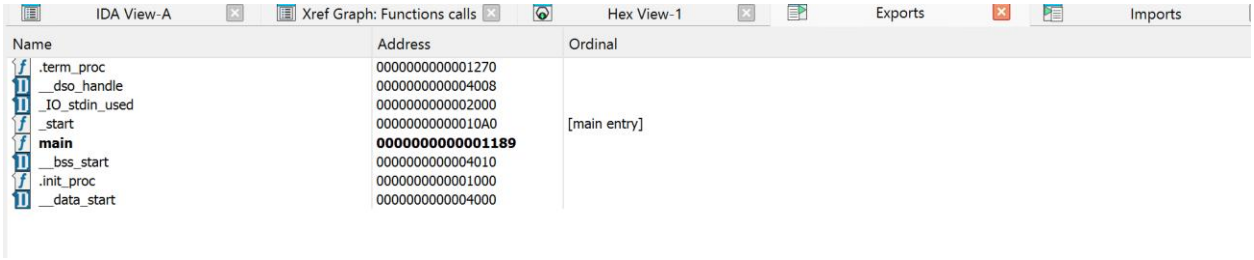


6. Captured a screenshot of the IDA View-A window. IDA View-A shows the actual machine-level instructions where input is taken, tax is calculated, and results are printed. The assembly instructions show that arithmetic operations are performed directly within the *main* function which increases the chances of a logical error in the computation.

7. Imports lists the external library functions used by the program. It includes *printf* and *__isoc99_scanf*, which are used for output and input operations. Also, *__libc_start_main* and *__stack_chk_fail* functions indicate program initialization and stack protection mechanisms.

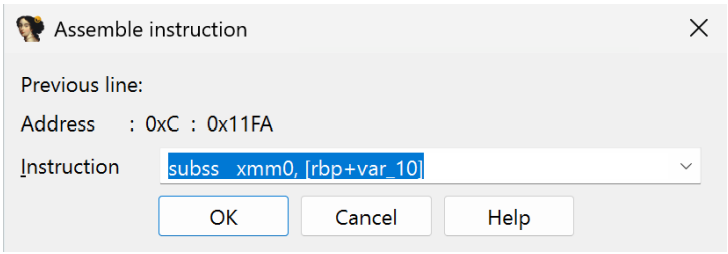


Exports lists functions defined within the binary itself. The *main* function serves as the entry point for the program. *_start, _init_proc*, and *_term_proc* functions are related to program initialization and termination.

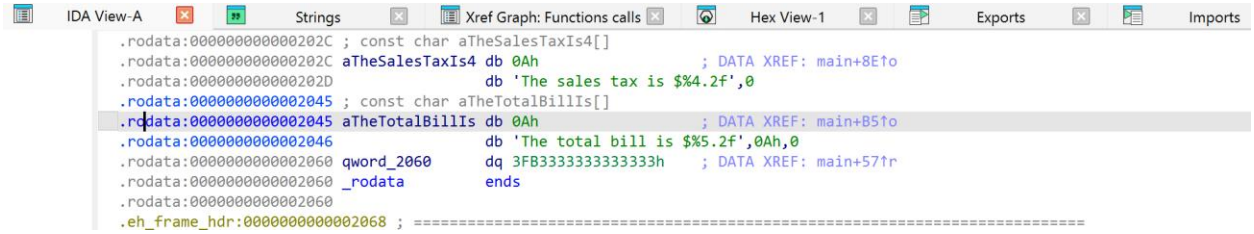


Part 2b - Fixing the Bug and Patching the Code

1. Navigated to *Edit >Patch Programs >Assemble* in IDA Pro. This will allow modification of the assembly instructions directly within the binary. This instruction says to subtract the calculated tax from the original purchase amount. This results in an incorrect total value.



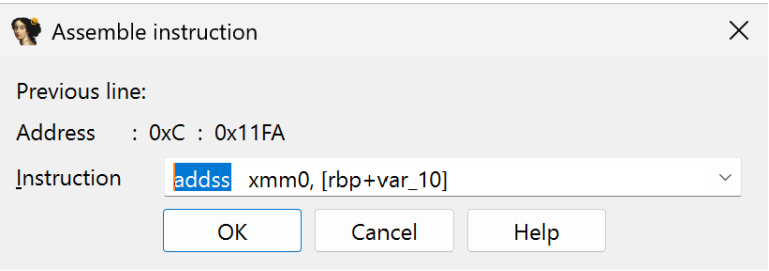
2. Navigated to *View >Open sub view >strings*. Double-clicked the string *The total bill is $%5.2f* to trace its cross-reference to the corresponding function where the total bill is calculated.



3. Double-clicked on the cross-reference of the string *The total bill is $%5.2f*, the program navigated to the *main* function where this string is used. This confirms that the calculation and display of the total bill are handled within the main function.

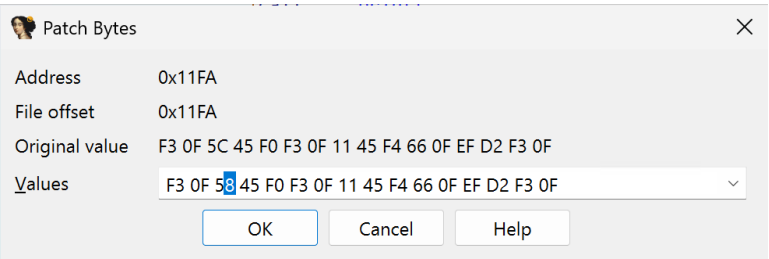
```
call    _printf
pxor    xmm3, xmm3
cvtss2sd xmm3, [rbp+var_C]
movq    rax, xmm3
movq    xmm0, rax
lea     rax, aTheTotalBillIs ; "\nThe total bill is $%5.2f\n"
mov     rdi, rax        ; format
```

4. Patched instruction replacing subtraction with addition. However, this method did not successfully modify the underlying machine code.



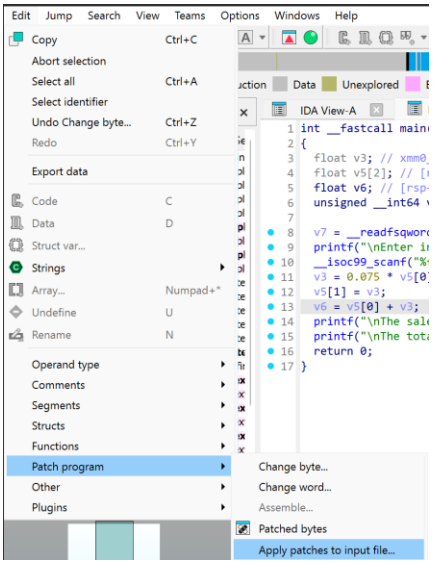

```
call    ___isoc99_scanf
movss   xmm0, [rbp+var_14]
pxor    xmm1, xmm1
cvtss2sd xmm1, xmm0
movsd   xmm0, cs:qword_2060
mulsd   xmm0, xmm1
cvtsd2ss xmm0, xmm0
movss   [rbp+var_10], xmm0
movss   xmm0, [rbp+var_14]
subss   xmm0, [rbp+var_10]
movss   [rbp+var_C], xmm0
```

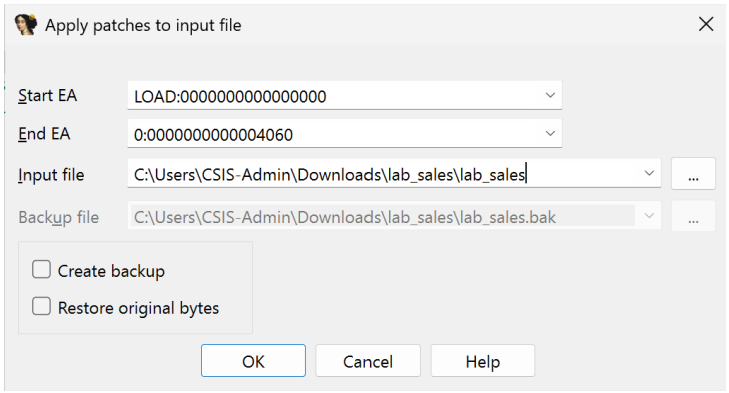
Then the patch was applied manually using the Patch Bytes feature. The Patch Bytes changed from 5C (SUBSS) to 58 (ADDSS). It correctly modified the instruction at the byte level.



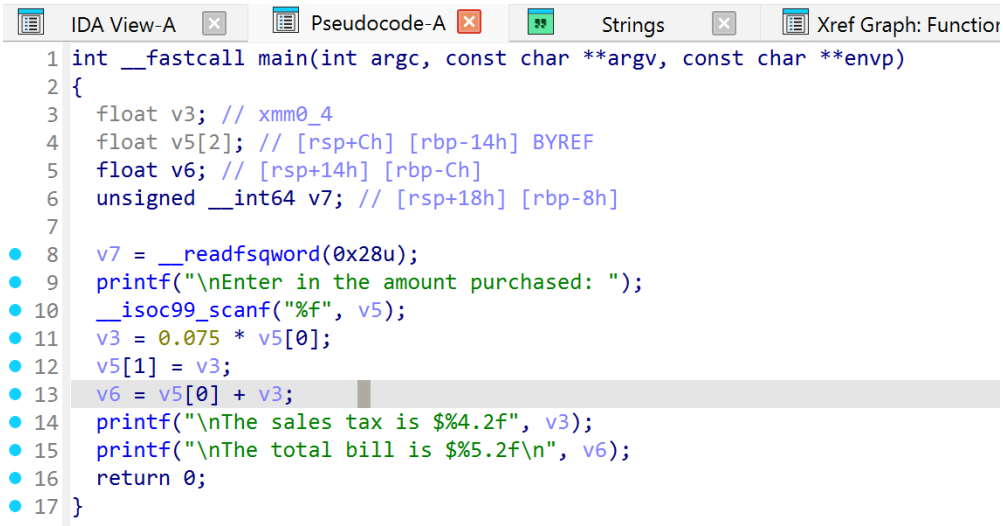

```
call    ___isoc99_scanf
movss   xmm0, [rbp+var_14]
pxor    xmm1, xmm1
cvtss2sd xmm1, xmm0
movsd   xmm0, cs:qword_2060
mulsd   xmm0, xmm1
cvtsd2ss xmm0, xmm0
movss   [rbp+var_10], xmm0
movss   xmm0, [rbp+var_14]
addss   xmm0, [rbp+var_10]
movss   [rbp+var_C], xmm0
```

To saved the patch by navigating to *Edit > Patch program > Apply patches to input file.*

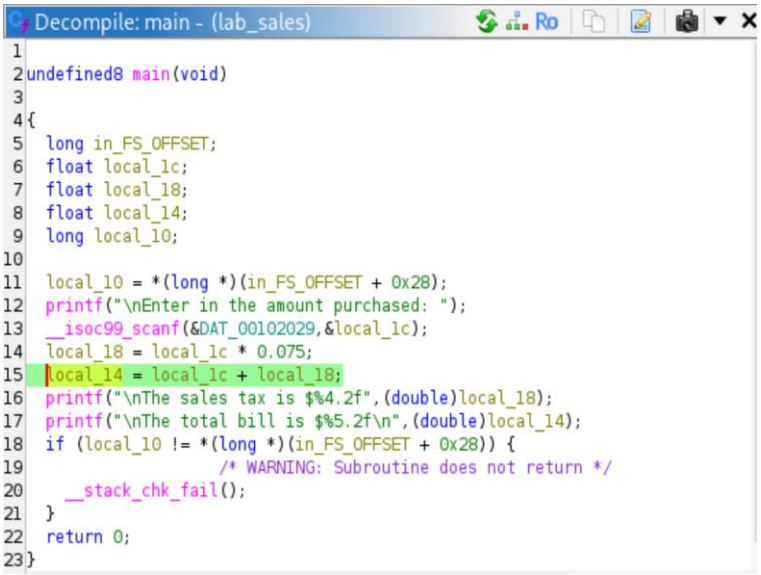




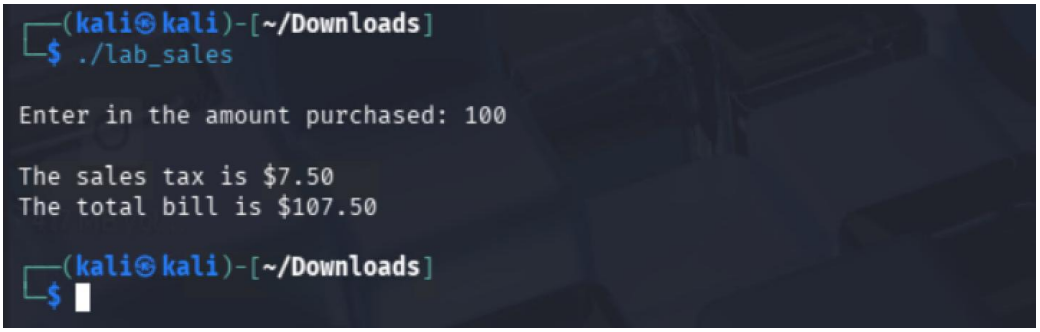
5. After applying the patch, checked the program in IDA Pro to verify that the issue was resolved. The pseudocode of *main* function now shows that the total bill is calculated using an addition operation (v6 = v5[0] + v3;) instead of subtraction.



6. Analyzed the patched file in Ghidra to verify the fix. The decompiled code shows that the total bill is now calculated using an addition operation (local_14 = local_1c + local_18;) instead of subtraction.



7. Tested the patched executable in the Kali Linux terminal by running the program and providing a sample input value of 100. The output showed that the sales tax was correctly calculated as 7.5% (7.50) and added to the original amount.

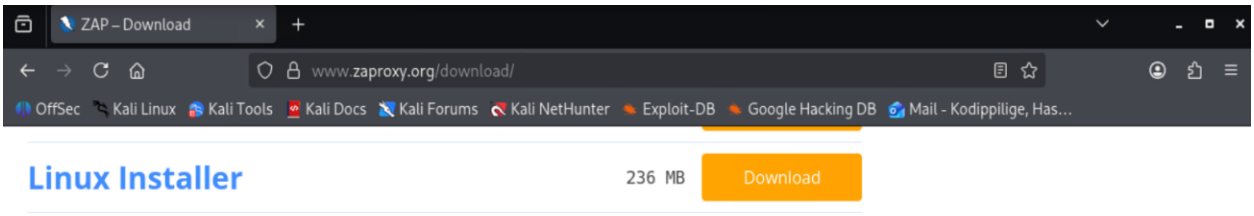


Part 3 - Vulnerability Assessment using OWASP ZAP Proxy

Objectives: To perform vulnerability assessment on a web application using a tool. To identify vulnerabilities and propose countermeasures.

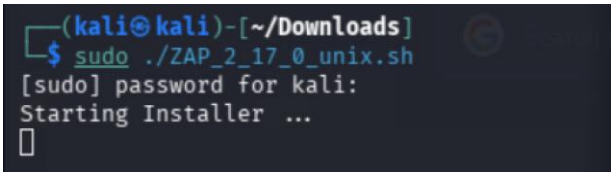
Part 3a

1. Inside the Kali, opened Firefox and navigated to the URL to download ZAP for Linux. Then downloaded the Linux Installer.

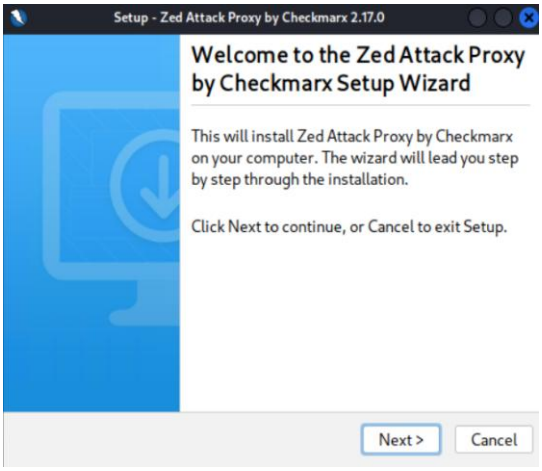


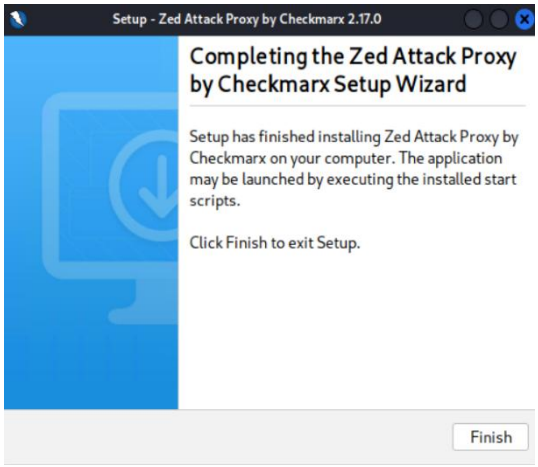
2. Changed the directory and navigated to the Download folder. The changed permissions for the downloaded file using the *chmod +x* command.

3. Next ran the *sudo ./ZAP_2_17_0_unix.sh* command to initiate the installion.

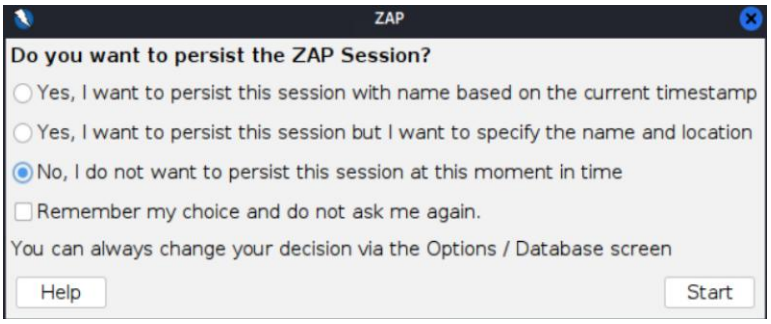


4. The Zed Attack Proxy (ZAP) Setup Wizard was launched. In the next window, accepted the terms and then accepted default settings and installed OWASP ZAP.

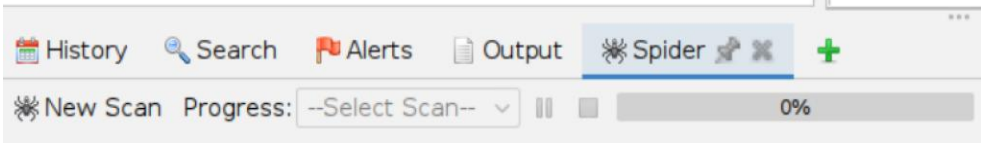




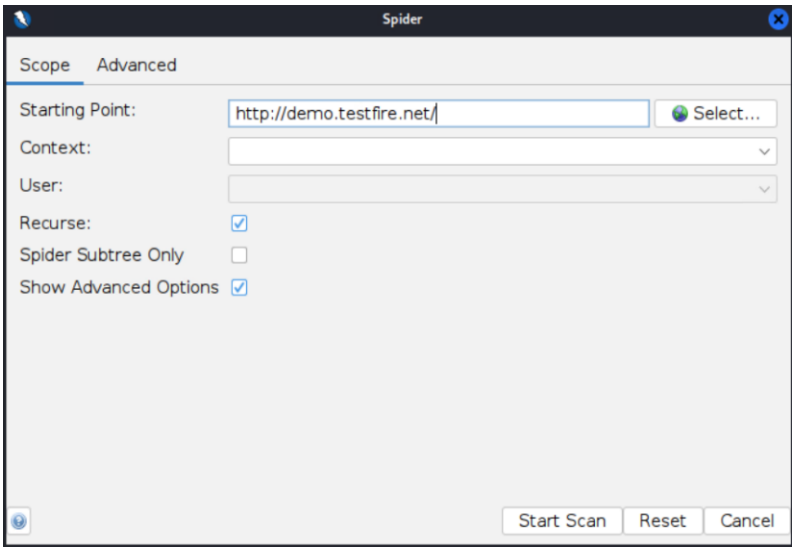
5. Launched ZAP to perform a passive scan from tool in Kali. Selected "No, I do not want to persist this session at this moment in time" and clicked Start.



6. At the bottom pane, right clicked on the green '+' sign and added the 'Spider' tool.

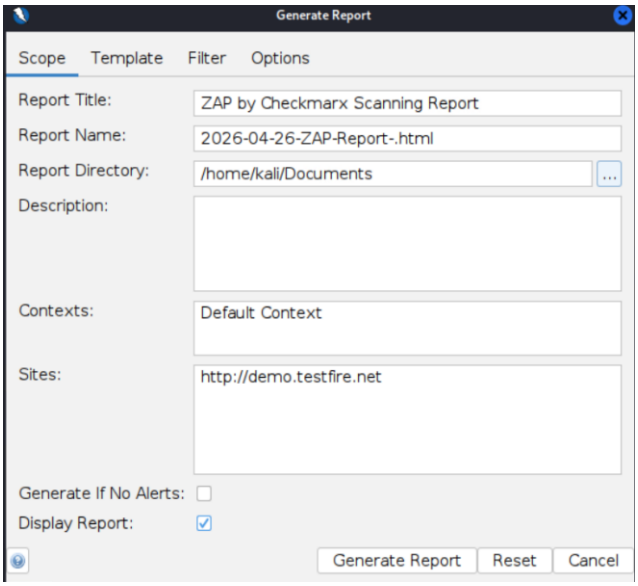


7. Clicked on Spider and then clicked on *New Scan*. Then entered URL http://demo.testfire.net/ in the starting point and clicked start scan.



8. After completing the scan, checked the Alerts by clicking *Alert* tab.

9. Navigated to the Report *Menu >Generate Report* to generate a report of scan. The report is attached to report as appendix A.



Part 3b - Vulnerabilities and Countermeasures

1. Absence of Anti-CSRF Tokens

This alert means a web application fails to verify that requests are intentionally sent by an authorized user. This will be vulnerable to Cross-Site Request Forgery (CSRF). Using this vulnerability, the attackers can trick authenticated users into performing unintended actions, such as changing passwords or submitting data, by clicking malicious links.

To mitigate this vulnerability, we can implement tokens in all forms and sensitive requests. Also, validate the token on the server side, use SameSite cookies to reduce cross-origin request risks, and require re-authentication for critical actions.

2. Content Security Policy (CSP) Header Not Set

This alert means our website lacks a crucial security layer to prevent Cross-Site Scripting (XSS), data injection, and malicious script execution.

To mitigate this vulnerability, we can configure the configure a Content Security Policy header (ex- Content-Security-Policy), restrict allowed sources for scripts, styles, and media, avoid using unsafe-inline and unsafe-eval, regularly review and update CSP rules.

3. Missing Anti-clickjacking Header

This alert means a website is vulnerable to having its content embedded in a hidden <*iframe*> on a malicious site. This trickery causes users to unknowingly click or take actions, enabling attacks like unauthorized transfers or data theft.

Using this vulnerability, attackers can load the application inside a hidden iframe and trick users into clicking malicious elements, leading to unauthorized actions.

Conclusion

This project helps to cover the reverse engineering and vulnerability assessment techniques to analyze and improve software security. Ghidra and IDA Pro can be used to examine compiled binaries and reconstruct their logic without access to the original source code. This process enabled to identify logical errors in the sales program which was calculating the total bill incorrectly. The issue was resolved by analyzing and patching assembly instructions. It highlights the importance of understanding low-level program behavior in debugging and software maintenance.

The OWASP ZAP provides valuable insights into common web application vulnerabilities. Scan results from this tool revealed several security issues, including the absence of Anti-CSRF tokens, missing Content Security Policy headers, and lack of clickjacking protection. These vulnerabilities show how web applications can be exposed to attacks if proper security controls are not implemented. Implementing security headers and input validation mechanisms are important as secure development practices.

This project covers the practical knowledge in binary analysis, debugging, and web security testing. This work highlights of identifying vulnerabilities early and applying effective fixes to ensure system reliability and security.

References

1. Álvarez Pérez, D., & Tiwari, R. (2025). *Ghidra software reverse-engineering for beginners* (2nd ed.). Packt Publishing.
2. Fox, N. (2023, April 6). *How to use Ghidra to reverse engineer malware*. Varonis. https://www.varonis.com/blog/how-to-use-ghidra
3. Siahaan, C. N. (2022, October 7). *Binary patching with IDA Pro (part 1)*. Medium. https://medium.com/@k_kisanak/binary-patching-with-ida-pro-part-1-c806d0f20d22
4. Minnesota State University Moorhead. (2026). *Installing Ghidra in Kali Linux*. D2L Brightspace. https://mnstate.learn.minnstate.edu/
5. Minnesota State University Moorhead. (2026). *Creating and running executable programs in Kali Linux*. D2L Brightspace. https://mnstate.learn.minnstate.edu/
6. Medium. (n.d.). *Securing binaries against reverse engineering: A developer's guide*. Medium. https://medium.com/
7. Zaproxy. (n.d.). *Getting started*. OWASP ZAP. https://www.zaproxy.org/getting-started/

ZAP by Checkmarx Scanning Report

Generated with The ZAP logoZAP on Sun 26 Apr 2026, at 19:03:00

ZAP Version: 2.17.0

ZAP by Checkmarx

Contents

About This Report

Report Parameters

Contexts

No contexts were selected, so all contexts were included by default.

Sites

The following sites were included:

- http://demo.testfire.net

(If no sites were selected, all sites were included by default.)

An included site must also be within one of the included contexts for its data to be included in the report.

Risk levels

Included: High, Medium, Low, Informational

Excluded: None

Confidence levels

Included: User Confirmed, High, Medium, Low

Excluded: User Confirmed, High, Medium, Low, False Positive

Summaries

Alert Counts by Risk and Confidence

This table shows the number of alerts for each level of risk and confidence included in the report.

(The percentages in brackets represent the count as a percentage of the total number of alerts included in the report, rounded to one decimal place.)

		Confidence				
		User Confirmed	**High**	**Medium**	**Low**	**Total**
Risk	**High**	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)
	Medium	0 (0.0%)	2 (18.2%)	1 (9.1%)	1 (9.1%)	4 (36.4%)
	Low	0 (0.0%)	1 (9.1%)	3 (27.3%)	0 (0.0%)	4 (36.4%)
	Informational	0 (0.0%)	0 (0.0%)	3 (27.3%)	0 (0.0%)	3 (27.3%)
	Total	0 (0.0%)	3 (27.3%)	7 (63.6%)	1 (9.1%)	11 (100%)

Alert Counts by Site and Risk

This table shows, for each site for which one or more alerts were raised, the number of alerts raised at each risk level.

Alerts with a confidence level of "False Positive" have been excluded from these counts.

(The numbers in brackets are the number of alerts raised for the site at or above that risk level.)

		Risk			
		High (= High)	**Medium (>= Medium)**	**Low (>= Low)**	**Informational (>= Informational)**
Site	**http://demo.testfire.net**	0 (0)	4 (4)	4 (8)	3 (11)

Alert Counts by Alert Type

This table shows the number of alerts of each alert type, together with the alert type's risk level.

(The percentages in brackets represent each count as a percentage, rounded to one decimal place, of the total number of alerts included in this report.)

Alert type	**Risk**	**Count**
Absence of Anti-CSRF Tokens	Medium	3 (27.3%)
Content Security Policy (CSP) Header Not Set	Medium	5 (45.5%)
Missing Anti-clickjacking Header	Medium	5 (45.5%)
Sub Resource Integrity Attribute Missing	Medium	1 (9.1%)

Cookie without SameSite Attribute	Low	3 (27.3%)
Cross-Domain JavaScript Source File Inclusion	Low	1 (9.1%)
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	5 (45.5%)
X-Content-Type-Options Header Missing	Low	5 (45.5%)
Information Disclosure - Suspicious Comments	Informational	5 (45.5%)
Modern Web Application	Informational	4 (36.4%)
Session Management Response Identified	Informational	3 (27.3%)
Total		11

Insights

This table shows information that is likely to be very relevant to you, but which is not related to vulnerabilities, or potentially even related to the application in question.

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP warnings logged - see the zap.log file for details	2
Info	Informational	http://demo.testfire.net	Percentage of responses with status code 2xx	48 %
Info	Informational	http://demo.testfire.net	Percentage of responses with status code 3xx	1 %
Info	Informational	http://demo.testfire.net	Percentage of responses with status code 4xx	49 %
Info	Informational	http://demo.testfire.net	Percentage of endpoints with content type application/javascript	2 %
Info	Informational	http://demo.testfire.net	Percentage of endpoints with content type application/pdf	1 %
Info	Informational	http://demo.testfire.net	Percentage of endpoints with content type application/x-shockwave-flash	1 %
Info	Informational	http://demo.testfire.net	Percentage of endpoints with content type image/gif	8 %
Info	Informational	http://demo.testfire.net	Percentage of endpoints with content type image/jpeg	28 %
Info	Informational	http://demo.testfire.net	Percentage of endpoints with content type image/png	2 %
Info	Informational	http://demo.testfire.net	Percentage of endpoints with content type text/css	2 %
Info	Informational	http://demo.testfire.net	Percentage of endpoints with content type text/html	50 %
Info	Informational	http://demo.testfire.net	Percentage of endpoints with method GET	96 %
Info	Informational	http://demo.testfire.net	Percentage of endpoints with method POST	3 %
Info	Informational	http://demo.testfire.net	Count of total endpoints	84
Info	Informational	http://demo.testfire.net	Percentage of slow responses	4 %

Alerts

1. **Risk=Medium, Confidence=High (2)**
    1. **http://demo.testfire.net (2)**
        1. Content Security Policy (CSP) Header Not Set **(1)**
            1. ▶ GET http://demo.testfire.net/sitemap.xml
        2. Sub Resource Integrity Attribute Missing **(1)**
            1. ▶ GET http://demo.testfire.net/index.jsp?content=personal_investments.htm

2. **Risk=Medium, Confidence=Medium (1)**
    1. **http://demo.testfire.net (1)**
        1. Missing Anti-clickjacking Header (1)
            1. ▶ GET http://demo.testfire.net/
3. **Risk=Medium, Confidence=Low (1)**
    1. **http://demo.testfire.net (1)**
        1. Absence of Anti-CSRF Tokens (1)
            1. ▶ GET http://demo.testfire.net/login.jsp
4. **Risk=Low, Confidence=High (1)**
    1. **http://demo.testfire.net (1)**
        1. Server Leaks Version Information via "Server" HTTP Response Header Field (1)
            1. ▶ GET http://demo.testfire.net/sitemap.xml
5. **Risk=Low, Confidence=Medium (3)**
    1. **http://demo.testfire.net (3)**
        1. Cookie without SameSite Attribute (1)
            1. ▶ GET http://demo.testfire.net/sitemap.xml
        2. Cross-Domain JavaScript Source File Inclusion (1)
            1. ▶ GET http://demo.testfire.net/index.jsp?content=personal_investments.htm
        3. X-Content-Type-Options Header Missing (1)
            1. ▶ GET http://demo.testfire.net/
6. **Risk=Informational, Confidence=Medium (3)**
    1. **http://demo.testfire.net (3)**
        1. Information Disclosure - Suspicious Comments (1)
            1. ▶ GET http://demo.testfire.net/login.jsp
        2. Modern Web Application (1)
            1. ▶ GET http://demo.testfire.net/index.jsp?content=inside.htm
        3. Session Management Response Identified (1)
            1. ▶ GET http://demo.testfire.net/sitemap.xml

Appendix

Alert Types

This section contains additional information on the types of alerts in the report.

1. **Absence of Anti-CSRF Tokens**

Source raised by a passive scanner (Absence of Anti-CSRF Tokens)
CWE ID 352
WASC ID 9
Reference
1. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
2. https://cwe.mitre.org/data/definitions/352.html

2. **Content Security Policy (CSP) Header Not Set**

Source raised by a passive scanner (Content Security Policy (CSP) Header Not Set)
CWE ID 693
WASC ID 15
Reference
1. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP
2. https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html
3. https://www.w3.org/TR/CSP/
4. https://w3c.github.io/webappsec-csp/
5. https://web.dev/articles/csp
6. https://caniuse.com/#feat=contentsecuritypolicy
7. https://content-security-policy.com/

3. **Missing Anti-clickjacking Header**

Source raised by a passive scanner (Anti-clickjacking Header)
CWE ID 1021
WASC ID 15
Reference
1. https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options

4. **Sub Resource Integrity Attribute Missing**

Source raised by a passive scanner (Sub Resource Integrity Attribute Missing)
CWE ID 345
WASC ID 15
Reference
1. https://developer.mozilla.org/en-US/docs/Web/Security/Subresource_Integrity

5. **Cookie without SameSite Attribute**

Source raised by a passive scanner (Cookie without SameSite Attribute)
CWE ID 1275
WASC ID 13
Reference
1. https://datatracker.ietf.org/doc/html/draft-ietf-httpbis-cookie-same-site

6. **Cross-Domain JavaScript Source File Inclusion**

Source raised by a passive scanner (Cross-Domain JavaScript Source File Inclusion)
CWE ID 829
WASC ID 15

7. **Server Leaks Version Information via "Server" HTTP Response Header Field**

Source raised by a passive scanner (HTTP Server Response Header)
CWE ID 497
WASC ID 13
Reference
1. https://httpd.apache.org/docs/current/mod/core.html#servertokens
2. https://learn.microsoft.com/en-us/previous-versions/msp-n-p/ff648552(v=pandp.10)
3. https://www.troyhunt.com/shhh-dont-let-your-response-headers/

8. **X-Content-Type-Options Header Missing**

Source raised by a passive scanner (X-Content-Type-Options Header Missing)
CWE ID 693
WASC ID 15
Reference
1. https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85)
2. https://owasp.org/www-community/Security_Headers

9. **Information Disclosure - Suspicious Comments**

Source raised by a passive scanner (Information Disclosure - Suspicious Comments)
CWE ID 615
WASC ID 13

10. **Modern Web Application**

Source raised by a passive scanner (Modern Web Application)

11. **Session Management Response Identified**

Source raised by a passive scanner (Session Management Response Identified)
Reference
1. https://www.zaproxy.org/docs/desktop/addons/authentication-helper/session-mgmt-id/